

Senatla Ops

Supabase and Vercel Guidance

Backend, runtime configuration and deployment guidance.

VERSION	1.0
DATE	23 June 2026
STATUS	Controlled release baseline
CLASSIFICATION	Confidential - internal use

Prepared by Sankofa Digital Studio for Senatla Trading

Table of contents

Recommendation	3
Why this pairing is the default	3
Better alternatives only if your priority changes	3
Environment variables	3
Supabase setup	3
Current backend scope	3
Next backend module	4

Recommendation

Use:

- Supabase for Auth, Postgres, RLS, and persistent operational state
- Vercel for frontend hosting and future thin privileged functions

Do not use Vercel as the primary data layer for this project. The domain is relational and audit-heavy. Supabase fits that better.

Why this pairing is the default

- payroll, attendance, sites, and audit data are relational
- browser-only persistence is no longer acceptable
- Supabase RLS gives a clean path to role-bound access rules
- Vercel hosts Angular cleanly and can add server functions later without replacing the data tier

Better alternatives only if your priority changes

Choose Clerk + Neon + Vercel instead if:

- auth complexity is the main problem
- you need stronger org/invite/session tooling before data modeling
- you expect enterprise identity requirements first

Do not choose Firebase first for this app unless:

- realtime mobile sync is more important than relational reporting and audit design

Environment variables

Set these for local .env usage and in Vercel project settings:

- SENATLA_API_MODE=supabase
- SENATLA_SUPABASE_URL=...
- SENATLA_SUPABASE_ANON_KEY=...

Use local mode when you want the current demo gateway path.

Supabase setup

1. Create a Supabase project. 2. Run the SQL migration in [20260406_init_senatla_ops.sql](C:\Users\thoso\Dev\Portfolio\senatla\senatla-ops\supabase\migrations\20260406_init_senatla_ops.sql). 3. Create Auth users in Supabase Auth. 4. Insert matching rows into public.profiles with:

- id = auth user id
- display_name
- username
- role

Current backend scope

Implemented now:

- Supabase-backed auth session restore and sign-in

- Supabase-backed persistent app snapshot storage
- runtime config generation for local and Vercel builds

Not implemented yet:

- immutable server-side audit writes
- row-level policies for per-site data slices inside normalized tables
- server-side payroll/export endpoints

Next backend module

The next correct backend step is:

- move audit events out of snapshot storage into immutable Supabase tables
- write those events through a trusted server path or insert-only RLS path